

PEP 8 — стиль Python-коду

Розгорнутий довідник із поясненнями «чому саме так»

PEP — Python Enhancement Proposal, документ-пропозиція щодо розвитку мови. Їх сотні, і **PEP 8** серед них — той, що описує оформлення коду. Написали його Гвідо ван Россум (автор Python), Баррі Ворсоу і Ніку Коглан.

PEP 8 — **не синтаксис**. Код із порушеннями запуститься. Це домовленість спільноти: якщо всі пишуть в одному стилі, будь-який чужий код читається як свій.

Головний принцип самого PEP 8: «A Foolish Consistency is the Hobgoblin of Little Minds» — сліпа послідовність шкідлива. Якщо правило робить код *менш* читабельним, або весь проєкт історично написано інакше — тримайся стилю проєкту. Читабельність важливіша за букву правила.

1. Відступи та перенесення рядків

1.1 Чотири пробіли на рівень

Табуляція заборонена в новому коді. Змішувати таби й пробіли в одному файлі Python 3 узагалі не дозволяє — це `TabError`.

Чому пробіли: ширина табу залежить від редактора (2, 4 або 8), тож один і той самий файл у різних людей виглядає по-різному. Пробіл скрізь однаковий. У PyCharm клавіша Tab уже вставляє 4 пробіли.

1.2 Продовження довгого рядка

Є два дозволені способи. **Вирівнювання за відкритою дужкою:**

```
result = some_function(first_argument, second_argument,
                        third_argument, fourth_argument)
```

Висячий відступ (hanging indent) — після дужки нічого не лишається, аргументи з наступного рядка:

```
result = some_function(
    first_argument,
    second_argument,
)
```

✗ **Аргументи на першому рядку при висячому відступі**

```
foo = long_function(var_one,
                    var_two, var_three)
```

✗ **Відступ тіла збігається з аргументами**

```
def long_function(
    var_one, var_two):
    print(var_one)
```

В останньому випадку додають зайвий рівень, щоб аргументи візуально відрізнялися від тіла функції:

```
def long_function(
    var_one, var_two):
    print(var_one)
```

1.3 Закриваюча дужка

Або під першим елементом, або на початку рядка — обирай одне й тримайся:

```
numbers = [
    1, 2, 3,
    4, 5, 6,
]
```

```
numbers = [
    1, 2, 3,
    4, 5, 6,
]
```

1.4 Дужки замість зворотного слеша

Рядок можна розірвати зворотним слешем `\`, але це крихко: пробіл після слеша — і код зламався, а помилку видно не одразу. Дужки безпечніші.



```
if (weather == "rain"
    and temperature < 10):
    take_umbrella()
```



```
if weather == "rain" \
    and temperature < 10:
    take_umbrella()
```

Слеш лишається доречним там, де дужок бути не може — наприклад, у старому `with` із кількома менеджерами.

1.5 Перенесення перед оператором

Довгу формулу розривають **перед** знаком `+`, `and`, `*`, а не після. Так у лівій колонці видно, що робить кожен рядок.



Рекомендовано (стиль Кнута)

```
total = (salary
        + bonus
        - tax)
```



Оператори ховаються в кінці

```
total = (salary +
        bonus -
        tax)
```

Історія: до 2016 року PEP 8 вимагав протилежного. Правило змінили, бо математики століттями пишуть саме так. Обидва варіанти досі допустимі — головне не змішувати їх в одному проєкті.

1.6 Довжина рядка

Що	Ліміт PEP 8	Як на практиці
Код	79 символів	Часто 88 (Black) або 99–120 у командах
Коментарі та docstring	72 символи	Лишають вужчими за код

Навіщо ліміт: два файли поруч на одному екрані; діф у code review без горизонтальної прокрутки; довгий рядок майже завжди означає, що в ньому забагато логіки. Коментарі роблять вужчими, бо суцільний текст читається краще вузькою колонкою — як у книжці.

2. Порожні рядки та структура файлу

2.1 Скільки порожніх рядків

Де	Скільки
Між функціями й класами верхнього рівня	2
Між методами всередині класу	1
Між логічними блоками всередині функції	1, за потреби
Після рядка <code>class Foo:</code> перед першим методом	0 (або 1, якщо є docstring)

```
import os

CONSTANT = 10

class Account:
    """Рахунок користувача."""

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance -= amount

def main():
    ...
```

Чому саме два: два порожні рядки на око відрізняються від одного, і межі функцій видно під час швидкої прокрутки, не читаючи тексту. Це єдине призначення правила.

2.2 Кодування файлу

Python 3 читає файли як **UTF-8** за замовчуванням. Рядок `# -*- coding: utf-8 -*-` — спадок Python 2, у новому коді він зайвий.

3. Імпорти

3.1 Три групи, розділені порожнім рядком

```
import os
import sys
from pathlib import Path

import requests
from django.db import models

from myapp.utils import helper
```

Порядок груп: **стандартна бібліотека** → **сторонні пакети** → **власний код проєкту**. Усередині групи — за алфавітом.

Чому саме так: за секунду видно, що проєкт тягне ззовні. Порожній рядок між групами — межа між «це є в кожного Python» і «це треба встановити через pip».

3.2 По одному модулю в рядку



```
import os
import sys
```



```
import os, sys
```

Виняток — форма `from`, там кілька імен в один рядок нормально: `from typing import List, Optional`.

3.3 Ніколи `from module import *`

Зірочка тягне у твій файл усі імена модуля. Наслідки: незрозуміло, звідки взялася функція; чужі імена мовчки перезаписують твої; лінтер і автодоповнення IDE перестають працювати.

```
from tkinter import *
from turtle import *      # хто з них дав color()? Невідомо.
```

3.4 Абсолютні імпорти



Однозначно

```
from myapp.models import User
```



Крихко

```
from ..models import User
```

Імпорти ставлять **угорі файлу**, після docstring модуля й до будь-якого коду.

4. Пробіли у виразах

4.1 Де пробілів бути не повинно



```
spam(ham[1], {eggs: 2})
print(x, y)
dct["key"] = value
if x == 4:
```



```
spam( ham[ 1 ], { eggs: 2 } )
print ( x , y)
dct [ "key" ] = value
if x == 4 :
```

- Одразу всередині дужок будь-якого виду;
- Перед комою, крапкою з комою, двокрапкою;
- Перед дужкою виклику функції або індексації.

4.2 Знак `=` — два різні випадки

Ситуація	Пробіли	Приклад
Присвоєння	так	<code>count = 10</code>
Аргумент за замовчуванням	ні	<code>def f(name="Ігор")</code>
Іменований аргумент у виклику	ні	<code>f(name="Ігор")</code>
Аргумент з анотацією типу	так	<code>def f(name: str = "Ігор")</code>

Логіка: без пробілів `f(name="Ігор")` читається як один цілісний аргумент. Але щойно з'являється анотація, аргумент стає довгим — і пробіли потрібні, щоб не злипалося.

4.3 Пріоритет операцій можна підкреслювати

Навколо бінарних операторів — по одному пробілу. Але у складній формулі дозволено притиснути те, що виконується першим:

✓ Пріоритет видно

```
y = a*x**2 + b*x + c
c = (a+b) * (a-b)
```

✗ Пробіли невпадають

```
y = a * x ** 2+b * x+c
c = (a + b)*(a - b)
```

4.4 Зрізи (slice)

Двокрапка у зрізі — оператор із однаковими пробілами обабіч; у простих випадках пробілів немає взагалі.

```
items[1:9]
items[lower:upper]
items[lower+offset : upper+offset]
items[: upper_fn(x) : step_fn(x)]
```

4.5 Інлайн-коментар — два пробіли перед

```
x = x + 1  # компенсація зсуву рамки
```

Після # — рівно один пробіл. Такий самий пробіл ставлять і в блокових коментарях.

4.6 Кома в кінці (trailing comma)

У багаторядкових структурах кому після останнього елемента ставлять свідомо: рядок додається без правки сусіднього, і діф у git показує один рядок замість двох.



```
colors = [
    "red",
    "green",
]
```

✗ В одному рядку — зайва

```
colors = ["red", "green",]
```

Пастка: `x = (1,)` — це кортеж з одного елемента, а `x = (1)` — просто число 1. Тут кома змінює тип, а не оформлення.

5. Іменування

5.1 Зведена таблиця

Об'єкт	Стиль	Приклад
Змінна, функція, метод, аргумент	snake_case	user_name , get_total()
Константа	UPPER_SNAKE_CASE	MAX_RETRIES
Клас, виняток	CapWords	UserAccount , ValueError
Модуль, пакет	коротко, малими	utils.py , models
Внутрішнє (не для зовнішнього використання)	_один підкреслювач	_cache
Атрибут класу з підміною імені	__два підкреслювачі	__secret
Обхід збігу з ключовим словом	підкреслювач у кінці	class_ , type_
Магічні методи	__dunder__	__init__ — свої не вигадують

5.2 Що означають підкреслювачі

- `_name` — **домовленість**: «це внутрішнє, ззовні не чіпай». Технічно доступ не блокується; `from module import *` такі імена пропускає.
- `__name` в класі — **name mangling**: Python перейменовує атрибут на `_ClassName__name`. Це не захист від злому, а захист від випадкового збігу імен у спадкоємця.
- `name_` — коли потрібне ім'я, зайняте мовою: `list_`, `id_`, `class_`. Ніколи не пишуть `class` чи `lst`.

5.3 Три заборонені імені

```
l = 10      # мала L
O = 20      # велика o
I = 30      # велика i
```

У більшості шрифтів `l` не відрізнити від `1`, а `O` — від `0`. PEP 8 забороняє їх як одиночні імена прямим текстом.

5.4 Перший аргумент

- Метод екземпляра — завжди `self`;
- Метод класу (`@classmethod`) — завжди `cls`.

Мова дозволяє будь-яке ім'я, але порушення цієї домовленості збиває з пантелику кожного читача.

5.5 Що передає добре ім'я

- Функція — **дієслово**: `calculate_total()`, `send_email()`;
- Змінна — **іменник**: `user`, `total_price`;
- Логічне значення — **питання**: `is_active`, `has_permission`;
- Колекція — **множина**: `users`, `prices`.

Довжина імені пропорційна області видимості: `i` в трирядковому циклі — нормально, `i` як глобальна змінна — ні.

6. Коментарі та docstring

6.1 Загальні вимоги

- Коментар, що суперечить коду, **гірший за його відсутність**. Змінив код — онови коментар;
- Пиши повними реченнями, з великої літери;
- Мова — англійська, якщо код може читати не лише твоя команда;
- Пояснюй **чому**, а не **що**: «що» видно з коду.

6.2 Docstring (PEP 257)

Публічні модулі, класи й функції мають рядок документації в потрібних лапках одразу після заголовка. Він доступний у `help()` і в підказках PyCharm — на відміну від звичайного коментаря.

```
def calculate_discount(price, percent):  
    """Повертає ціну зі знижкою.  
  
    price: початкова ціна, percent: знижка у відсотках (0-100).  
    Кидає ValueError, якщо percent поза діапазоном.  
    """  
    if not 0 <= percent <= 100:  
        raise ValueError("percent має бути від 0 до 100")  
    return price * (1 - percent / 100)
```

Для однорядкового docstring закриваючі лапки лишають на тому ж рядку. У багаторядковому — на окремому.

7. Рекомендації щодо самого коду

7.1 Порівняння з `None` — тільки через `is`



```
if value is None:  
    if value is not None:
```



```
if value == None:  
    if not value is None:
```

`None` існує в єдиному екземплярі, тож правильне питання — «це той самий об'єкт», а не «рівні за значенням». До того ж клас може перевизначити `__eq__` і зробити `== None` істинним.

7.2 Порожні послідовності перевіряють напряму



```
if not items:  
    if names:
```



```
if len(items) == 0:  
    if len(names) > 0:
```

Хибними в Python вважаються: `False`, `0`, `0.0`, `""`, `[]`, `{}`, `()`, `None`.

Обережно: якщо `0` — коректне значення, скорочення бреше. `if count:` пропустить нуль так само, як і `None`. У таких місцях пиши `if count is not None:` явно.

7.3 Не порівнюй з `True` і `False`

```
if is_ready == True:      # зайве  
if is_ready is True:     # ще й крихко
```

```
if is_ready:  
    if not is_ready:
```

7.4 Префікси й суфікси рядків



```
if name.startswith("test_"):   
if file.endswith(".py"):
```



```
if name[:5] == "test_":  
if file[-3:] == ".py":
```

Зріз доводиться перераховувати вручну щоразу, коли міняється підрядок, — і саме там з'являються помилки на одиницю.

7.5 Тип перевіряють через `isinstance`

```
if isinstance(value, int):
```

```
if type(value) == int:
```

`isinstance` враховує спадкування, тому працює з підкласами. `type() ==` їх відкидає.

7.6 `def` замість присвоєння `lambda`



```
def square(x):  
    return x * x
```



```
square = lambda x: x * x
```

У варіанті з `lambda` функція лишається безіменною: у трасуванні помилки буде `<lambda>` замість `square`. Сама по собі `lambda` доречна як аргумент: `sorted(data, key=lambda p: p.age)`.

7.7 try — мінімальний, except — конкретний



```
try:
    value = int(text)
except ValueError:
    value = 0
```



```
try:
    value = int(text)
    save(value)
    send_report()
except:
    pass
```

Голий `except:` коштує навіть `KeyboardInterrupt` — програму стає неможливо зупинити `Ctrl+C`. Якщо потрібно спіймати все, пиши `except Exception:`. А зайві рядки всередині `try` призводять до того, що ти обробляєш геть не ту помилку, на яку розраховував.

7.8 Узгоджений return

Якщо функція десь повертає значення, вона має повертати його на **всіх** шляхах — інакше в одній із гілок мовчки повернеться `None`.



```
def get_bonus(sales):
    if sales > 100:
        return sales * 0.1
    return 0
```



```
def get_bonus(sales):
    if sales > 100:
        return sales * 0.1
    # тут неявний None
```

7.9 Власні винятки — від Exception

Не від `BaseException`: від нього успадковані системні `KeyboardInterrupt` і `SystemExit`, які ловити не можна. Назва класу закінчується на `Error`, якщо це саме помилка:

```
class PaymentError(Exception):
    """Не вдалося провести платіж."""
```

8. Анотації типів

Оформлення анотацій описує **PEP 484**, а пробіли в них — PEP 8. Це підказки для людини й IDE: Python їх **не перевіряє** під час виконання.

```
def greet(name: str, times: int = 1) -> str:
    return f"Привіт, {name}! " * times
```

- Двокрапка притиснута до імені, після неї — пробіл: `name: str`;
- Стрілка `->` з пробілами обабіч;
- Значення за замовчуванням при анотації — з пробілами навколо `=`.

9. Інструменти замість ручної праці

Інструмент	Що робить
PyCharm , Ctrl+Alt+L	Форматує відкритий файл за PEP 8. Хвилясте підкреслення — уже знайдене порушення
flake8	Класичний перевіряльник: показує порушення, але не виправляє
black	Форматує без питань і налаштувань. Ліміт рядка — 88
ruff	Сучасна заміна flake8 + black, у сотні разів швидша
isort	Сортує імпорти по групах

```
pip install ruff
ruff check .      # знайти порушення
ruff format .     # виправити оформлення
```

Межа можливостей автоматик: інструменти розставляють пробіли й переноси. **Імена змінних, зміст коментарів і структуру функцій** вони не чіпають — а це і є найважливіша частина читабельності. Форматер прибирає суперечки про дрібниці, щоб на code review лишалося обговорення суті.

10. Коли PEP 8 можна порушити

Сам документ називає такі випадки:

- Дотримання правила робить код **менш** читабельним;
- Решта проєкту історично написана інакше — послідовність усередині проєкту важливіша;
- Код має лишитися сумісним зі старішою версією Python, яка потрібного синтаксису не розуміє;
- Правило суперечить зовнішньому інтерфейсу, який ти не контролюєш.

Порядок пріоритетів у спірному випадку: **послідовність усередині функції** → **усередині файлу** → **усередині проєкту** → **PEP 8**. І ніколи не ламай зворотну сумісність заради краси.

Повний текст: peps.python.org/pep-0008 · Docstring: PEP 257 · Анотації типів: PEP 484